

Metrics in Agile Project Courses

Lukas Alperowitz
Technical University of Munich
Munich, Germany
alperowi@in.tum.de

Dora Dzvonyar
Technical University of Munich
Munich, Germany
dzvonyar@in.tum.de

Bernd Bruegge
Technical University of Munich
Munich, Germany
bruegge@in.tum.de

ABSTRACT

We believe that software engineering should be taught in a hands-on way such as through a project-based capstone course where students apply the learned concepts in a real setting. However, such a teaching format can be challenging and time-consuming for instructors.

In this paper we explain how we selected and introduced a set of metrics to improve the manageability of our large multi-project capstone course. We regularly run such a course with over 100 students developing applications in 10-12 parallel projects over the course of one semester.

Our approach focuses on measuring the success of three key workflows, namely Merge Management, Continuous Integration and Continuous Delivery. We show how these metrics help the instructors to keep track of the progress of multiple projects running at the same time, enabling them to identify and react to problems early.

Categories and Subject Descriptors

K.6.3 [Management Of Computing And Information Systems]: Software Management

Keywords

agile software engineering, capstone course, project management, metrics, continuous integration, continuous delivery

1. INTRODUCTION

We believe that software engineering should be taught with a hands-on approach, which means that the students apply the theory they learned in a real setting. Therefore, we create a realistic setting in our capstone courses, in which students develop applications in teams for real customers [3]. Managing such a project-based software engineering course with multiple projects running at the same time is rather challenging for instructors. Complexity increases even further if state-of-the-art software engineering

workflows like Continuous Delivery or asynchronous code reviews are added to the syllabus [12]. The instructor has to introduce these workflows to students with different backgrounds in software engineering, and then continuously ensure that the students are able to successfully apply the learned concepts and techniques in their projects. This can be very time-consuming, which is why the instructor needs a way to quickly assess each project's status and identify potential problems which call for action.

We propose the usage of metrics to increase the manageability of such a software engineering project course. We regularly run a capstone course with over 100 students developing applications in 10-12 parallel projects with customers from industry over the course of three months [1]. During the summer term of 2015, we selected and evaluated metrics for the key workflows of our course. In this paper we present our approach and explain the benefits and challenges of using metrics to evaluate the progress of student projects.

We start with an overview of the project course and then establish goals for selecting a set of metrics. We then describe our workflows and discuss how we selected metrics to measure their adaptation and usage. Our focus hereby is on three key workflows, namely Merge Management, Continuous Integration and Continuous Delivery. We describe our experiences with using these metrics for the management of our project course in a case study we conducted in the summer term of 2015. Finally, we discuss the results and provide an outlook on the next steps we want to take.

2. COURSE STRUCTURE

In this section we describe the structure and organization of a multi-customer capstone course we conduct regularly. We started teaching similar capstone courses in 2008 and since then have continuously improved our approaches and workflows. In this course, over 100 students work in teams over the period of 3 months to develop mobile applications and the corresponding systems for more than 10 partners from industry. The course is based on our process model named Rugby [3], which is a version of Scrum [13] adapted to account for part-time developers.

Figure 1 shows the structure of the course using a simplified organizational chart. As shown in the figure, two instructors are responsible for the overall management of the course. Each project team, represented as a vertical bar, consists of customer, project manager, coach and students who form the development team. The *customer* is an employee of a participating industry partner and takes the role of the Product Owner as defined in Scrum [13]. The *project*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE '16 Companion, May 14-22, 2016, Austin, TX, USA

© 2016 ACM. ISBN 978-1-4503-4205-6/16/05...\$15.00

DOI: <http://dx.doi.org/10.1145/2889160.2889183>

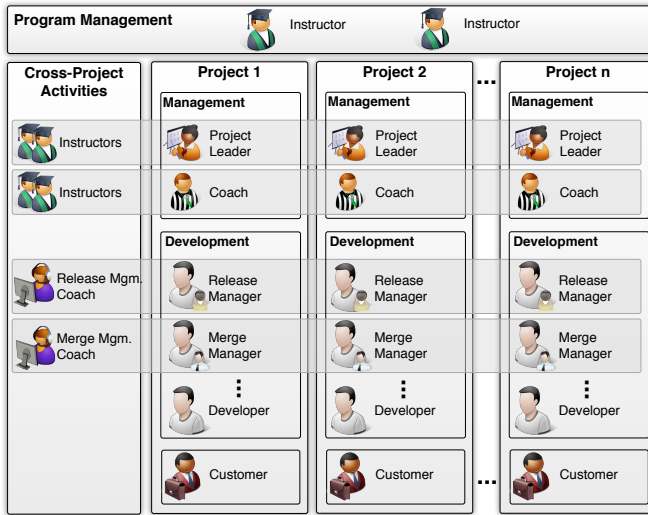


Figure 1: Organizational chart of the course [3]

manager is a graduate teaching assistant in the role of a Scrum Master. A *coach* is a more experienced student who has already participated in the course as a developer. The coach supports the project manager in his day-to-day tasks. Some of the developers are part of a cross-project team [3], which means that this developer takes over the responsibility for certain topics such as Continuous Delivery in the team. We describe the course organization in more detail in [3].

In this multi-project organization, each development team works independently from the other teams on solving a problem for their customer. They often work in very different ways due to vastly different project characteristics which demand individual approaches [7]. Both the *high number* of simultaneous projects and the *varying nature* of the projects make it considerably difficult for instructors to assess the progress of the individual teams. It is especially challenging to determine how well the workflows are understood and adopted in the individual project teams.

We propose the use of a set of metrics to enable the instructors need a high-level overview over the projects to recognize problems as early as possible. Our metrics also allow them to drill down and investigate the root causes if the indicators suggest a problem in the project.

3. SELECTION OF METRICS

In this section we describe how we selected and defined the metrics for three key workflows of our capstone course. We first discuss the goals and explain the target audience. Afterwards we describe each of the three workflows and the metrics we selected to measure its adoption and usage.

As a project-based organization our capstone course is run by instructors and students in multiple roles, which we described in detail in section 2. We think that choosing the right target audience for the metrics upfront is an important success factor. While instructors and project leaders are teaching assistants, team coaches and developers are students and therefore much less experienced in project management. We think that students who have not yet experienced how a project's characteristics can influence such indicators would tend to interpret the metrics too literally,

while instructors have the proper mindset to use the metrics as a basis for making decisions. For instance, making the number of commits of each team member available to the participants might drive them to increase their commit count by e.g. making unnecessary commits, because many of them fail to understand the limited explanatory power of this metric when considered in isolation. Moreover, we think that students will see the metrics as the basis of their final grade in the course although we deliberately emphasize less quantifiable factors such as team work and leadership abilities as the basis of our final assessment.

In addition to limiting the use of metrics for direct comparisons *within* teams, we do not think that comparing whole teams directly *to each other* is useful to the outcome of the course. Each team works on a different project with its particular challenges, and they can take very different paths to reach their goal [7]. Our experience shows that a team's characteristics ranging from the type of problem to solve to the personality of the team members can be quite different, and they can and should not be ranked directly against each other using performance indicators. For instance, a team can have a large number of releases because they are dealing with a customer who constantly gives feedback on the development progress, but that does not necessarily mean that their progress is better than another team with less releases that is dealing with a busy customer who only finds time for the project every some weeks. Thus, we do not want to constrain a project team in the way they apply our workflows, but prefer to enable the team to organize themselves according to their specific needs.

Considering the above-described points, we came up with the following goals for selecting a set of metrics:

- We want to create metrics solely for the use of instructors and project leaders, and therefore strictly do not expose them to students participating in the course.
- We want to provide a high-level overview over the individual projects which allows the instructors and project leaders to recognize and react to problems early.
- We want to help instructors to more easily focus their attention on teams where the adoption of certain workflows is falling short of their expectations.

To demonstrate the applicability of our approach, we focus on three core workflows we teach to the participants, namely Merge Management, Continuous Integration and Continuous Delivery. In the following sections, we briefly describe each workflow and explain how we selected key metrics to measure its adoption in the project teams. During the selection process, we analyzed each of our workflows with regard to its desired outputs and selected metrics based on the techniques suggested by [11, 9, 8].

3.1 Merge Management

In our course we use a simplified version of the gitflow branching model [4]. We teach students to develop each new feature on a separate branch, a so-called *feature branch*. After finishing a feature, the developer integrates it into the application by merging his feature branch into the development branch. Before a feature branch is merged, we introduced a quality gate in the form of merge requests. This allows us to conduct a code review in which experienced

team members review the code and give suggestions in form of textual comments or tasks to solve for improvements. The workflow and the branching model are described in detail in [3].

It is important to us instructors that the team members get used to the Merge Management workflow as early as possible in order to minimize the organizational overhead and provide for better code quality. However, we noticed two recurring problems that arise when merge requests are created by unexperienced developers. Firstly, feature branches tend to exist for a long time and can take days before they are finally merged, especially in the beginning of the project when developers are not yet familiar with the workflow. The reasons for this range from slow team communication to a developer not understanding the proposed changes given in the feedback he received on his proposed code changes. This is problematic because the larger the lifetime of a merge request, the higher the chances of encountering merge conflicts, as the development branch already contains a newer version of the code.

The second recurring problem we noticed is that the project teams sometimes tend to *just approve* a merge request without reviewing the code changes. This happens especially with increasing time pressure as the project progresses, e.g. in the days before the final presentations. Circumventing the quality gate introduced by the Merge Management workflow endangers the code and product quality, which is why we want to detect and react to this behavior.

Taking these recurring problems into account, we defined two metrics which seem appropriate for measuring the adoption of the Merge Management workflow in our project teams.

Metric: Merge Requests - Lifetime

We measure the average lifetime of a merge request between creation of the request and the point at which it was merged. We assumed an optimal lifetime to be under 24 hours, based on our experience from past courses and taking into account that participating students also have to fulfill other duties in their studies. However, a very short lifetime is also not desirable, as it can point to a lack of thoroughness when reviewing the code as described above. In addition to showing the average lifetime, we also provide an overview of all merge requests of the current and past weeks grouped by lifetime. This illustrates how the lifetime evolved during the course of the project and to see if feedback given by e.g. the instructor was taken into account by the team.

Metric: Merge Requests - Workflow

In addition to the lifetime of merge requests, we measured the usage of the Merge Management workflow. To evaluate if teams are reviewing code thoroughly, we counted the percentage of merge requests with at least one comment or task created during the current week. Both comments and tasks are important means for the reviewer to give feedback on the code changes before allowing the developer to merge these changes into the project's main codebase. Of course, the simple fact of having a comment in each merge request does not necessarily indicate that the workflow is well implemented by the team, but a high percentage of merge requests with no comment should catch the instructors' attention.

3.2 Continuous Integration

A key element of our course is a Continuous Delivery workflow which is explained in detail in [3]. As part of this workflow we use a central Continuous Integration server which runs a build and executes Unit-Tests as well as automated UI-Tests after each commit in a team's repository. If an error occurs, the author of the commit is notified, so that he can fix the code and the according build as soon as possible.

Experiences from past courses show that the reasons for a failing build can be manifold, ranging from coding errors over configuration errors of the build server to encoding issues. As Continuous Integration focuses on the key principle of getting the build running again as soon as possible, we measure this workflow with the following two metrics.

Metric: Continuous Integration - Time to fix

We calculate the time to fix, which is the average period of time between a failed build and the first successful build on the team's main development branch [6]. A high time to fix could e.g. indicate that a team does not really care about the status of the integration server, or that the developers are not experienced enough to fix challenging build errors.

Similarly to the merge request lifetime, we also measure the development of this indicator compared to the past weeks. If the time to fix increased dramatically, for instance, the instructor can take a look at the corresponding project's build plans to find out more about the root cause.

Metric: Continuous Integration - Builds

By showing the absolute numbers of successful and failed build we provide the instructor with further insights into the Continuous Integration workflow of each team. A large amount of failed builds could e.g. indicate that the team was unable to interpret the errors provided by the integration server or that the integration server itself did not work as expected.

3.3 Continuous Delivery

Based on Continuous Integration we also apply the concept of Continuous Delivery [10], allowing a project team to deliver each successfully tested build to their team members or customer. We encourage teams to deliver a new product increment to their customer regularly in order to define and refine their requirements, e.g. when they implemented a new feature. Our experience shows that early prototypes not only facilitate the communication between developers and customer, but also help to evaluate design choices right from the beginning of the project [7].

Metric: Continuous Delivery - Delivery to customer

With this indicator, we determine each week if any customer has downloaded a release of the team's application at least once. While we currently do not measure if the customer has used and tested the application, a successful download is a good indicator of the grade of his involvement in the project progress. If a team does not have any download by their customer for weeks, an instructor can inquire about possible root causes. These can range from a problem in communication with the customer to delivery problems due to the nature of the project. For instance, if the team develops a complex system with various sensors or other hardware devices involved, a delivery to the customer can be challenging.

Metric: Continuous Delivery - Downloads

We encourage the team to test a build of their system on their own devices before they deliver it to the customer. For this, we provide them with a simple workflow to download any successful build of their application to their devices. This allows them to minimize the amount of errors that can happen during the delivery of a new product increment [10]. We measure the number of downloads to team members' devices to determine if they make use of this workflow.

4. CASE STUDY

During the summer term of 2015, we created a reporting sheet¹ with the selected metrics and introduced it into our capstone course described in section 2. We combined these metrics on a single sheet for each project so that they can be evaluated in combination, because we believe that a single indicator can not accurately predict project success [11].

In our capstone course in 2014, more than 100 students developed applications in 11 projects with partners from industry over a period of 3 months. We generated the reporting sheet for each project team on a weekly basis during the whole duration of our course and used it to identify projects where certain workflows were not adopted as expected. After taking a more detailed look to identify possible root causes, we talked to the project leader or coach of the team to find a solution. Multiple times we found that even though some indicators pointed towards a problem, the team had very good reasons to work they way they did. In these cases we stuck to one of the key principles of the agile manifesto: "Individuals and interactions over processes and tools" [2], and did not force the teams to change their approach.

The instructors and project leaders were satisfied with the metrics sheet because it gave them a quick overview over the project status. If they noticed an irregularity, e.g. a high number of failed builds, they could go into the respective systems and investigate the root causes or talk to the responsible members of the development team. Therefore, the metrics were often a good first indicator, and proved to be a powerful tool to oversee several projects at once.

This first case study provides us with anecdotal evidence for the usefulness of metrics for instructors of large project courses. In the 2016 capstone course, we will conduct a controlled experiment to further analyze the influence of metrics in project courses and gather more robust data. We will use the results to further refine our approach and the corresponding set of metrics.

5. CONCLUSION

In this paper we showed a targeted way to support the teaching efforts of instructors of large capstone courses by using metrics. Metrics allow the instructors to maintain an overview over the projects and intervene when problems arise. We believe that the metrics described increase the manageability of such project courses.

Nevertheless, we think that regardless of how thoroughly the data is collected, a simple number should never be the only indicator of project progress when it comes to teaching software engineering in a real-world setting. We therefore

suggest to use these metrics with caution, especially when it comes to grading and assessing students' performance. Instructors should treat the metric as a first indicator of a possible problem and always investigate the root causes by e.g. speaking to the team members.

In the future, we will gather statistically more robust data on the utility of metrics for project management and education by conducting controlled experiments. We want to examine whether these metrics are suitable to assess project progress and how well they uncover the problems that can arise in a project. In addition we want to cover additional workflows and add more sources of data by e.g. including communication metrics to measure intra- and extra-team communication as described in [5].

6. ACKNOWLEDGMENTS

We thank all participants of the project courses as well as the teaching assistants who introduced and the workflows described in this paper. A special thanks to Jan Ole Johanssen and Daniel Menke for the creation of the reporting sheets.

7. REFERENCES

- [1] L. Alperowitz, Dora Dzvonyar, and B. Bruegge. iOS Praktikum SS 2015, July 2015. <http://www1.in.tum.de/ios15>.
- [2] K. Beck, M. Beedle, et al. Manifesto for agile software development. *The Agile Alliance*, 2001.
- [3] B. Bruegge, S. Krusche, and L. Alperowitz. Software engineering project courses with industrial clients. *ACM Transactions on Computing Education (TOCE)*, 15(4):17, 2015.
- [4] V. Driessen. A successful Git branching model, 2010. <http://bit.ly/1TghAwL>.
- [5] A. H. Dutoit and B. Bruegge. Communication metrics for software development. *IEEE Transactions on Software Engineering*, 24(8):615–628, 1998.
- [6] P. M. Duvall, S. Matyas, and A. Glover. *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley, 2007.
- [7] D. Dzvonyar, S. Krusche, and L. Alperowitz. Real projects with informal models. In *MODELS'14 - Proceedings of the 10th Educators' Symposium*, 2014.
- [8] N. E. Fenton and M. Neil. Software metrics: roadmap. In *Proceedings of the Conference on the Future of Software Engineering*, pages 357–370. ACM, 2000.
- [9] R. B. Grady. Successfully applying software metrics. *Computer*, 27(9):18–25, 1994.
- [10] J. Humble and D. Farley. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [11] C. Kaner et al. Software engineering metrics: What do they measure and how do we know? In *Proceedings of METRICS'04*, 2004.
- [12] S. Krusche and L. Alperowitz. Introduction of Continuous Delivery in Multi-customer Project Courses. In *Companion Proceedings of the 36th International Conference on Software Engineering*, pages 335–343. ACM, 2014.
- [13] K. Schwaber and M. Beedle. *Agile Software Development with Scrum*. Prentice Hall PTR, Upper Saddle River, NJ, USA, 1st edition, 2001.

¹An example reporting sheet is available at https://www1.in.tum.de/alperowitz/icse16_metrics